tween 10-100 depending on the model (Table 3 in appendix). These results show the crucial role that LayerNorm and RMSNorm play in stabilizing the intermediate hidden vectors in practice.

3.2 Rotation

While stabilizing the growing norm of hidden vectors is a major function of layer normalization, it also ends up rotating the hidden vectors. Figure 3 shows the angle between the original hidden vector and the post-normalization vector. As can be seen in Figure 3 (and in Figure 9 for the rest of the models in the appendix), each layer is responsible for a fixed, non-trivial amount of rotation of hidden vectors in the representation space. Mean rotation angles in degrees between hidden and post-layer normalization vectors (LN1 and LN2) across all layers for various models are shown in Table 4 in the appendix. Thus, both LayerNorm and RMSNorm additionally rotate hidden vectors in representation space in addition to stabilizing the norms of the vectors.

4 LayerNorm versus RMSNorm

Apart from norm stabilization and rotation, a critical aspect of LayerNorm is its ability to orient the hidden vectors orthogonal to the uniform vector, as discussed in section 2. In section 2.2, we see that the definition of LayerNorm is innately linked with the uniform vector, possibly unintended by the original authors. Since the information along the uniform vector is being removed irreversibly during layer normalization (section 2.3), the layer normalization process implicitly assumes that the information along the uniform vector is either not important or that the model should not store information along that direction. However, two randomly chosen vectors in high-dimensional spaces are nearly orthogonal to each other with a very small spread. Since the “mean subtraction" step in LayerNorm means removing the component along the arbitrarily chosen direction of the uniform vector, the need for such a step would be justified if the hidden representations created by the model had unnaturally significant components along this vector. However, there seems to be no justification given in the literature for the mean subtraction step, including the original paper (Ba et al., 2016).

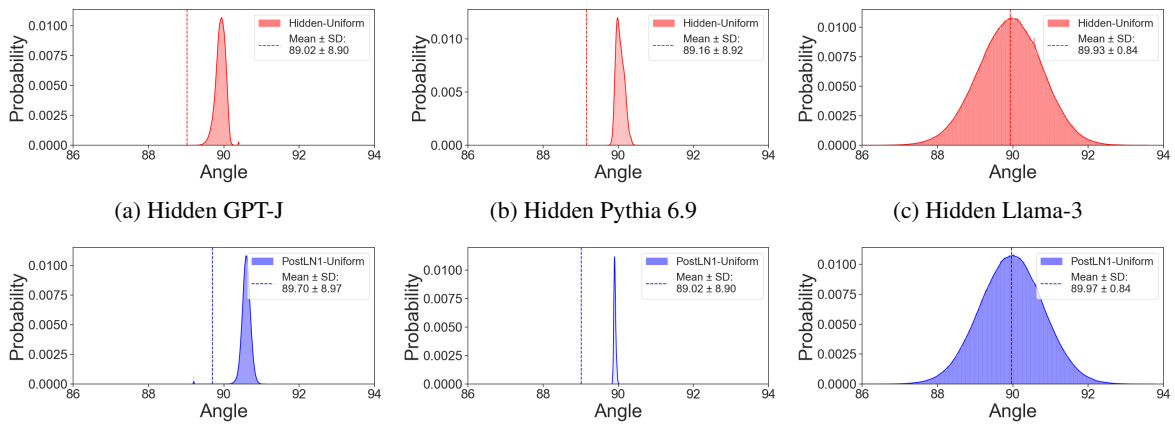
RMSNorm, on the other hand, does not perform this step and simply normalizes the existing vector (Zhang and Sennrich, 2019). The Llama model series (Touvron et al., 2023a,b) is the most prominent family of LLMs that use RMSNorm. The simple fact that Llama models achieve state-of-the-art results across multiple measures (Dubey et al., 2024) shows that using RMSNorm instead of LayerNorm does not hurt performance. This provides a strong motivation to explore the need for the “mean subtraction" step in LayerNorm. Thus, we ask the question - do intermediate hidden representations have a non-trivial component along the uniform vector that justifies its removal?

We study this question at inference time. To answer this question, we measure the angle between the hidden vector and the uniform vector just before and after the normalization operations for LayerNorm and RMSNorm-based models. To justify the mean subtraction step, one of the following two scenarios should be true:

- Scenario-1: In LayerNorm-based models, the intermediate hidden vectors have a large component along the uniform vector pre-normalization, which gets removed post-normalization.
- Scenario-2: In RMSNorm-based models, which are trained without the mean subtraction step, the hidden vectors consistently have a large component along the uniform vector.

The results are shown in Figure 5 for GPT-J (6B), Pythia (6.9B), and Llama-3 (8B) and Figure 11 for the remaining models. We see that for all models in Figure 5, the angle between the hidden vectors and the uniform vectors is 90 degrees on average even before normalization, with a very small spread. This remains true post-normalization as well for all three models. This result is surprising because the hidden representations for LayerNorm-based models (GPT-J and Pythia) are themselves orthogonal to the uniform vector even before the LayerNorm operation. This is true for 4 out of the 5 LayerNorm-based models studied in this paper except for GPT2-XL (Figure 11 in the appendix). This shows that the “mean subtracting" operation of LayerNorm is redundant during inference, as there is nothing to remove.

For RMNSNorm-based LLMs, the result for Llama-3 (8B) in Figure 5 represents scenraio -2. We see that even for Llama-3 (8B) trained using RMSNorm, the hidden representations before normalization are also on average orthogonal to the uniform vector with a very small spread. In fact,



(a) Hidden GPT-J (b) Hidden Pythia 6.9 (c) Hidden Llama-3

(d) Post-LN1 GPTJ (e) Post-LN1 Pythia 6.9 (f) Post-LN1 Llama-3

Figure 5: Distribution of angles (in degrees) between Hidden vectors (a-c) and post-normalization vectors (d-f) with the *uniform vector* for GPT-J, Pythia 6.9, Llama-3 for a randomly selected layer (Layer 24). The results are independent of the choice of layers.

they are more orthogonal to the uniform vector than for LayerNorm-based models, with a much smaller variance. After RMSNorm, the hidden representations continue to be orthogonal to the uniform vector. This is also true for Llama-2 (7B) (Figure 11 in the appendix). This shows that even without the "mean subtraction" step during training as in RMSNorm, the hidden vectors operate orthogonal to the uniform vector, underscoring the redundance of the "mean subtraction" step.

These results show that the "mean subtracting" operation in LayerNorm is redundant as the model naturally aligns its representations orthogonal to the uniform vector as expected in such high-dimensional spaces. In fact, in hindsight, evidence of having large components along the uniform vector would have been more surprising and indicated something fundamentally important about the specific vector. With this, we provide the first mechanistic evidence that the **"mean subtraction" step in LayerNorm is dispensable**, as we do not find hidden vectors having large components along the uniform vector. This line of investigation was made possible by our novel geometrical interpretation of LayerNorm (section 2.2), which presented the global implications of the mean subtraction step in LayerNorm - removing the component of intermediate hidden representations along the uniform vector. Due to these results, we advocate for using RMSNorm over LayerNorm, which is also computationally more efficient and leads to comparable downstream performance.

5 Conclusion

We present a detailed theoretical and empirical analysis of layer normalization on hidden vectors in representation space, with a focus on the global effects of the LayerNorm operation on a vector. We first show that the LayerNorm operation can be understood in three simple steps: removing the component of a vector along the uniform vector ($\mathbf{1} = [1, 1, 1, 1, \cdots, 1]^T \in \mathbb{R}^d$), normalizing the remaining vector, and scaling the resultant vector by $\sqrt{d}$. We then show that LayerNorm is an irreversible process—information along the uniform vector is removed during LayerNorm and cannot be recovered using the learnable parameters available in the formulation. This is in contrast with BatchNorm, where the network has the option of learning an identity operation. We then empirically illustrate how LayerNorm regulates the norm and the orientation of hidden vectors during model inference. Finally, we show that the "mean subtraction" operation in LayerNorm is dispensable both during inference and for training, as shown by internal representations of RMSNorm-based Llama models.

6 Limitations

This study provides valuable insights into the theoretical and empirical effects of layer normalization. Although the study empirically tests 7 models across two sizes and multiple language model families, the size of the models tested is restricted to 8 billion parameters. Since we wanted our results to have a large enough sample size, we chose to store

hidden representations of 1 million tokens. Even for a small model like GPT2-XL, this requires 2TB of data, and for larger models like Llama3-8B, it requires storing approximately 4TB of data. Storing such large amounts of hidden representations pushed the capacity of the computational resources available to us. Because of this restriction, we leave testing our findings for larger models to future works and groups that have access to larger amounts of computing.

References

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. arXiv preprint arXiv:1607.06450.

Shaked Brody, Uri Alon, and Eran Yahav. 2023. On the expressivity role of layernorm in transformers' attention. arXiv preprint arXiv:2305.02582.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. arXiv preprint arXiv:2407.21783.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778.

Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In International conference on machine learning, pages 448–456. pmlr.

Zixuan Jiang, Jiaqi Gu, Hanqing Zhu, and David Pan. 2024. Pre-rmsnorm and pre-crmsnorm transformers: equivalent and efficient pre-ln transformers. Advances in Neural Information Processing Systems, 36.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023a. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023b. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. Advances in neural information processing systems, 30.

Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin. 2019. Understanding and improving layer normalization. Advances in neural information processing systems, 32.

Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. Advances in Neural Information Processing Systems, 32.

A Appendix

A.1 Computations in Decoder-only LLMs

In this paper, we study the hidden representations of modern decoder-only large language models (LLMs). Let h^l represent the intermediate hidden state vectors between each decoder layer. As depicted in Figure 6, the computations within a layer of most decoder-only LLMs proceed as follows:

$$f^l = \texttt{LN1}(h^{l-1}) \tag{12}$$

$$a^l = \texttt{Att}(f^l) \tag{13}$$

$$g^l = \texttt{LN2}(h^{l-1} + a^l) \tag{14}$$

$$m^l = W^l_{proj}\sigma(W^l_{fc}g^l + b^l_{fc}) + b_{proj} \tag{15}$$

$$h^l = h^{l-1} + a^l + m^l \tag{16}$$

The intermediate hidden vectors between each layer, h^l, are also sometimes called the *residual stream*. Let LN1 represent the first LayerNorm function that acts just before the attention module and LN2 represent the second LayerNorm just before the MLP module. We abstract out the computations of the attention and MLP modules to focus on the layer normalization blocks.

As the vectors computed in the attention and MLP modules get added to the residual stream at each layer, the residual stream represents a summation of an increasing number of vectors. A non-recursive formula for the residual stream depicts this clearly:

$$h^l = h^0 + \sum_{i=0}^{i=l} a^i + \sum_{i=0}^{i=l} m^i \tag{17}$$

The residual stream thus represents a continuous summation of vectors computed at the attention and MLP modules in each layer with the incoming residual stream. This leads to an increasing norm of the residual stream (section 3.1), thus necessitating a normalization operation.